BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC FPT — FPTU QUY NHON AI CAMPUS

FU CAHATA · KHOA CÔNG NGHỆ THÔNG TIN — BỘ MÔN KỸ THUẬT PHẦN MỀM

OMNICAST MOBILE | ỨNG DỤNG DI ĐỘNG NATIVE FLUTTER

BÁO CÁO NGHIỆM THU ĐỒ ÁN HỌC PHẦN

HỌC PHẦN: PRM393 — PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

ỨNG DỤNG DI ĐỘNG PHÁT SÓNG VÀ TRUYỀN HÌNH TRỰC TUYẾN OMNICAST MOBILE

Phát sóng. Kết nối. Khám phá.

Clean Architecture 4 tầng, BLoC; LiveStream và Mobile Live Studio WebRTC WHIP; SQLite offline-first; OmniPlayer

Giảng viên hướng dẫn (Mentor):	**Thầy Huỳnh Vang** (Bộ môn Kỹ thuật Phần mềm — FPTU)
Đơn vị & Câu lạc bộ:	**FPTU Quy Nhon AI Campus — FU Cahata**
Học kỳ thực hiện:	Fall 2026 (FA26)
Thành viên nhóm (4 TV):	**1. Nguyễn Bình An** (Nhóm trưởng / Kiến trúc Backend & Bảo mật) **2. Trần Thanh Sơn** — MSSV: `QE170080` (gRPC Audit & Cơ sở dữ liệu) **3. Nguyễn Vĩnh Phú** — MSSV: `QE180136` (Trưởng nhóm Mobile Flutter) **4. Đồng Hoàng Nguyên** (Full-Stack Next.js, LiveStream & AI Curator)
Kho mã nguồn (GitHub):	https://github.com/TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester
Bản phát hành APK:	https://github.com/TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester/releases/tag/v1.2.0
Quy mô & Kiểm định:	**53.922 dòng mã Dart | 31 màn hình | 91 test trong test tree; CI run xác nhận 77/77**

FPTU QUY NHON AI CAMPUS — FU CAHATA · THÁNG 10 NĂM 2026

Phát triển ứng dụng di động đa nền tảng: Ứng dụng phát thanh truyền hình và phân phối nội dung số OmniCast Mobile

Nguyễn Bình An, **Trần Thanh Sơn** (QE170080), **Nguyễn Vĩnh Phú** (QE180136), **Đồng Hoàng Nguyên**

Giảng viên hướng dẫn (Mentor): **Thầy Huỳnh Vang**
FPTU Quy Nhon AI Campus — FU Cahata, Khoa Công nghệ Thông tin, Trường Đại học FPT
Báo cáo tích hợp môn PRM393

***Abstract*— OmniCast Mobile là ứng dụng di động phân phối nội dung số và phát thanh truyền hình trực tiếp (LiveStream) đa nền tảng được xây dựng tại FPTU Quy Nhon AI Campus — FU Cahata cho môn học PRM393 dưới sự hướng dẫn của thầy Huỳnh Vang bằng Flutter 3.x và ngôn ngữ Dart. Ứng dụng tích hợp kiến trúc phân tầng Clean Architecture 4 lớp kết hợp mô hình quản lý trạng thái luồng phản ứng BLoC với 12 module nghiệp vụ và 31 màn hình chuyên biệt. Bốn đóng góp kỹ thuật trọng tâm bao gồm: (1) trình phát đa năng `OmniPlayer` tích hợp luồng phát trực tiếp độ trễ thấp WebRTC/HLS từ MediaMTX, phân giải luồng trực tiếp Google Drive, tự động khám phá máy chủ nội bộ qua Wi-Fi (`LanDiscoveryService`) và phòng xem tương tác thời gian thực hai chiều (`LiveWatchScreen`: Live Chat phân nhánh, biểu cảm hạt nổi xúc giác 0ms, quyên góp Super Chat 6 mệnh giá, thông báo ghim từ Host và chuyển chương trình tức thời); (2) bàn điều khiển `LiveStudioScreen` 4 tab cùng máy quay tác nghiệp hiện trường `MobileStudioRecorderScreen` hỗ trợ đẩy sóng WebRTC WHIP trực tiếp, mã thời gian SMPTE, đồng hồ âm lượng Stereo VU Meter và máy nhắc chữ Teleprompter HUD; (3) động cơ ngoại tuyến (offline-first) trên nền cơ sở dữ liệu SQLite cục bộ với hàng đợi phát lại thao tác bất đồng bộ (`pending_ops`) và thông báo nhắc lịch chính xác trước 15 phút; cùng (4) hệ thống điều hướng tập trung GoRouter quản lý 31 màn hình với cơ chế bảo vệ phiên đăng nhập chặt chẽ. Test tree có 91 ca; GitHub Actions run 40630318005 thực thi 77 ca trong phạm vi workflow và cả 77 đều đạt.**

***Index Terms*—Flutter, Dart, BLoC, LiveStream, Live Studio, WebRTC WHIP, SQLite, offline-first, MediaMTX, HLS, GoRouter, FPTU Quy Nhon AI Campus, FU Cahata.**

I. Mở đầu

Trong kỷ nguyên phân phối nội dung số hiện đại, ứng dụng di động đã trở thành điểm tiếp xúc chính yếu giữa người dùng và các dịch vụ truyền thông trực tuyến. Khác với môi trường máy tính để bàn (web desktop) có đường truyền mạng ổn định và nguồn năng lượng dồi dào, môi trường di động đặt ra những thách thức kỹ thuật nghiêm ngặt: độ trễ mạng biến động liên tục, kết nối thường xuyên bị gián đoạn khi chuyển vùng (cell tower handover / Wi-Fi to cellular), hạn chế về bộ nhớ RAM và thời lượng pin thiết bị.

Để giải quyết toàn diện các thách thức trên, bài báo này trình bày thiết kế và hiện thực ứng dụng di động **OmniCast Mobile** — một ứng dụng phát thanh truyền hình, truyền phát trực tiếp (LiveStream) và phân phối nội dung số đa nền tảng được nghiên cứu và phát triển tại **FPTU Quy Nhon AI Campus — FU Cahata** cho môn học **PRM393 — Mobile Applications Development** dưới sự hướng dẫn chuyên môn của giảng viên (*Mentor*) **Thầy Huỳnh Vang**. Ứng dụng được xây dựng trên nền tảng Flutter 3.x với ngôn ngữ Dart (53.922 dòng mã Dart), áp dụng kiến trúc phân tầng Clean Architecture 4 lớp kết hợp mô hình quản lý trạng thái BLoC (Business Logic Component) điều phối 31 màn hình và 22 thành phần giao diện chuyên dụng.

A. Động lực và bài toán kỹ thuật

Quá trình xây dựng ứng dụng di động OmniCast tập trung vào việc giải quyết năm bài toán kỹ thuật cốt lõi:

1) **Khả năng hoạt động khi mất mạng (Offline-First):** Người dùng thường xuyên tra cứu lịch phát sóng (EPG) và danh sách theo dõi (Watchlist) trong điều kiện mất kết nối. Ứng dụng phải hỗ trợ lưu trữ cục bộ bằng cơ sở dữ liệu SQLite, xếp hàng các thao tác phát sinh khi ngoại tuyến và tự động phát lại (replay) đồng bộ lên máy chủ khi có mạng trở lại.
2) **Luồng phát đa phương tiện độ trễ thấp và tự động khám phá mạng LAN:** Hệ sinh thái OmniCast phát sóng qua nhiều giao thức: WebRTC / HLS thông qua MediaMTX, luồng trực tiếp Google Drive và luồng nhúng video ngoài. Trình phát `OmniPlayer` phải tự động quét và khám phá địa chỉ IP máy chủ MediaMTX trong mạng Wi-Fi cục bộ (`LanDiscoveryService`) và tự phục hồi khi luồng video bị đứt quãng.
3) **Phòng xem LiveStream tương tác hai chiều & Bàn điều khiển Mobile Live Studio:** Người xem không chỉ thụ động tiếp nhận luồng phát mà còn tham gia sảnh chờ, gửi bình luận phân nhánh, thả biểu cảm hạt nổi có phản hồi xúc giác 0ms, nhận thông báo ghim từ đạo diễn và quyên góp Super Chat 6 mệnh giá. Đồng thời, phóng viên hiện trường cần một bàn điều khiển `LiveStudioScreen` 4 tab và ống kính máy quay `MobileStudioRecorderScreen` hỗ trợ đẩy sóng WebRTC WHIP trực tiếp từ điện thoại.
4) **Quản lý trạng thái phân tán và bảo mật phiên:** Việc điều hướng 31 màn hình với 12 module BLoC yêu cầu cơ

Bảng I
DANH SÁCH 4 THÀNH VIÊN NHÓM DỰ ÁN OMNICAST (FPTU QUY NHON AI CAMPUS — FU CAHATA, MENTOR: THẦY HUỲNH VANG).

Họ và tên	Vai trò / MSSV	Phụ trách kỹ thuật chính
Nguyễn Bình An (`TomOutfit`)	Nhóm trưởng	Kiến trúc lõi NestJS 19 module, Bảo mật JWT & RBAC 4 vai trò, Thuật toán chống xung đột lịch EPG, Direct Stream Engine, CI/CD & Triển khai
Trần Thanh Sơn	`QE170080`	Kiến trúc vi dịch vụ gRPC Audit Logger (`audit_logger.proto`), Cơ sở dữ liệu PostgreSQL Supabase (21 Prisma models), Telemetry & Seed
Nguyễn Vĩnh Phú	`QE180136`	Trưởng nhóm Mobile Flutter (31 màn hình, 12 BLoC), Trình phát `OmniPlayer`, `MobileStudioRecorderScreen` (WebRTC WHIP), Offline SQLite
Đồng Hoàng Nguyên (`teng122`)	Full-Stack & AI	Cổng Web Next.js 15 Cinema Dark (33 route), Cổng Socket.IO `/live`, Phòng Live 7 chủ đề, Bàn điều khiển Live Studio & Module AI Curator

chế kiểm soát trạng thái chặt chẽ, cùng với việc lưu trữ token JWT an toàn trong Keystore/Keychain để tự động gia hạn phiên làm việc ngầm mà không làm phiền người dùng.

B. Tổ chức nhóm nghiên cứu và Phân công nhiệm vụ

Dự án được thực hiện bởi nhóm 4 sinh viên tại **FPTU Quy Nhon AI Campus — FU Cahata** dưới sự hướng dẫn của giảng viên **Thầy Huỳnh Vang**, như trình bày tại Bảng I.

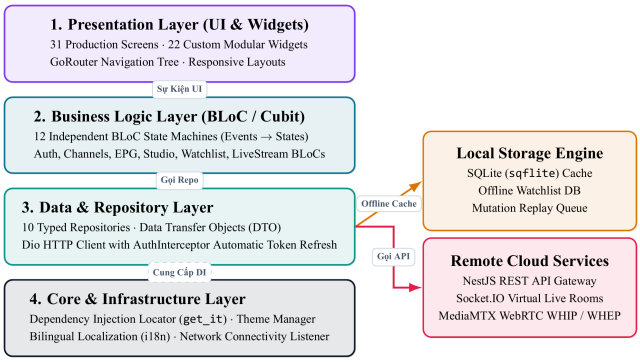
C. Đóng góp chính của báo cáo

Báo cáo này phân tích chi tiết năm đóng góp công nghệ của ứng dụng:

- Kiến trúc Clean Architecture 4 tầng (Presentation, Logic/ BLoC, Data, Core) với cơ chế Service Locator qua `get_it`.
- Hệ sinh thái `OmniPlayer`, `LiveWatchScreen`, `LiveStudioScreen` (4 tab) và máy quay `MobileStudioRecorderScreen` (WebRTC WHIP, SMPTE Timecode, Stereo VU Meter, Teleprompter HUD).
- Mô hình quản lý trạng thái luồng (Stream-based BLoC) với các concurrency transformer như `droppable` và `restartable`.
- Động cơ ngoại tuyến SQLite với hàng đợi `pending_ops` và cơ chế tự động giải quyết xung đột dữ liệu.
- Test tree có 91 kiểm thử; CI run 40630318005 thực thi 77 kiểm thử trong phạm vi workflow và cả 77 đều đạt.

II. KIẾN TRÚC ỨNG DỤNG DI ĐỘNG

OmniCast Mobile tuân thủ triệt để nguyên lý phân tách trách nhiệm (Separation of Concerns) và kiến trúc Clean Architecture được điều chỉnh cho hệ sinh thái Flutter. Cấu trúc mã nguồn được phân chia thành bốn tầng độc lập: Presentation, Logic (BLoC), Data và Core, bảo đảm tính độc lập khi kiểm thử (testability), khả năng tái sử dụng mã nguồn và tính dễ bảo trì.



Hình 1. Sơ đồ kiến trúc phân tầng 4 lớp của ứng dụng di động OmniCast: tầng Presentation điều hướng qua GoRouter (31 màn hình, 22 widget), tầng Logic với 12 BLoC/Cubit, tầng Data giao tiếp đồng thời với REST API, Socket.IO `/live` và SQLite ngoại tuyến.

A. Phân tầng trách nhiệm

Bốn tầng kiến trúc được tổ chức như sau:

1) **Presentation Layer:** Gồm 31 màn hình (screens) và 22 thành phần giao diện tái sử dụng (widgets). Tầng này hoàn toàn không chứa logic nghiệp vụ phức tạp mà chỉ chịu trách nhiệm hiển thị trạng thái và phát sinh các sự kiện người dùng (User Events).
2) **Logic Layer (BLoC / Cubit):** Gồm 12 module điều phối nghiệp vụ. BLoC tiếp nhận sự kiện từ Presentation, giao tiếp với tầng Data qua các Repository và phát ra các trạng thái mới (States) thông qua các Stream bất đồng bộ.
3) **Data Layer:** Chứa 10 Repository, Data Sources và 10 Model dữ liệu. Tầng này trừu tượng hóa nguồn gốc dữ liệu: tự động quyết định đọc từ bộ nhớ đệm SQLite hay gọi mạng từ xa qua Dio Client.
4) **Core Layer:** Cung cấp các tiện ích dùng chung: bộ định tuyến `GoRouter`, bộ tiêm phụ thuộc `get_it`, dịch vụ khám phá máy chủ phát sóng nội bộ `LanDiscoveryService`, dịch vụ đẩy luồng `WebRTCPublisherService`, cấu hình giao diện (theme), chuỗi bản địa hóa (i18n), dịch vụ thông báo cục bộ và quản lý token bảo mật.

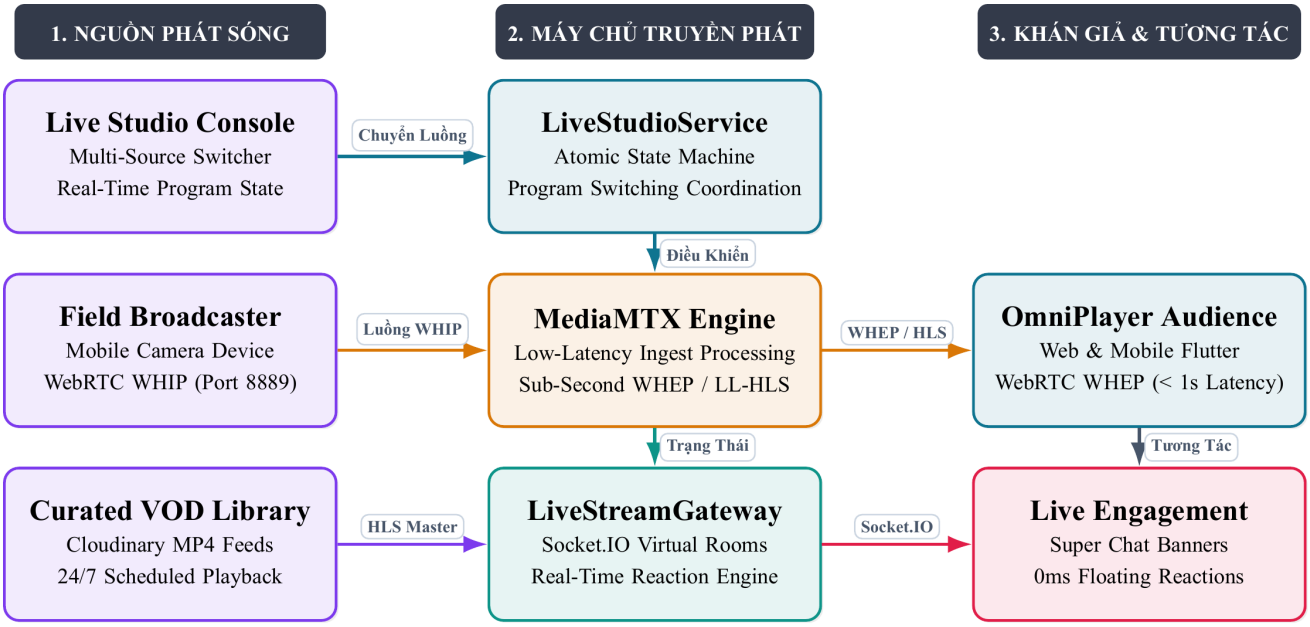
B. Bộ tiêm phụ thuộc (Dependency Injection)

Toàn bộ các thực thể Repository, Data Source, HTTP Client và BLoC được khởi tạo và quản lý vòng đời thông qua thư viện `get_it`. Trong tệp `injection_container.dart`, cơ chế đăng ký được phân định rõ ràng giữa Lazy Singleton (cho các dịch vụ mạng, cơ sở dữ liệu và repository) và Factory (cho các BLoC cần tạo mới theo vòng đời màn hình):

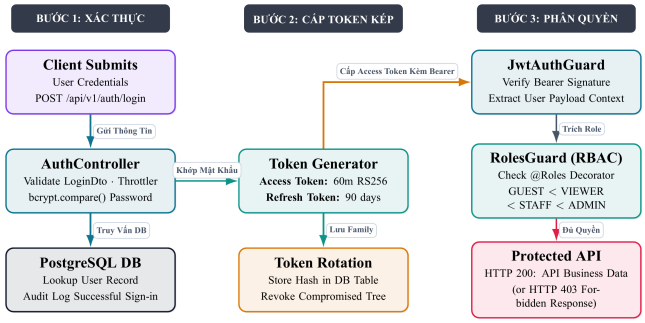
```dart
final sl = GetIt.instance;

Future<void> init() async {
  // BLoCs (Factory)
  sl.registerFactory(() => AuthBloc(sl()));
  sl.registerFactory(() => WatchlistBloc(sl()));
  sl.registerFactory(() => EpgBloc(sl()));

  // Repositories (Lazy Singleton)
```



Hình 2. Kiến trúc truyền phát trực tiếp (LiveStream) và bàn điều khiển Live Studio đầu-cuối giữa ứng dụng di động Flutter, cổng Web Next.js 15, máy chủ MediaMTX (WHIP/WHEP/HLS) và cổng Socket.IO `/live`.



Hình 3. Quy trình xác thực JWT phân tầng và kiểm soát truy cập RBAC 4 cấp độ kết nối giữa Flutter Client và NestJS API.

```
  sl.registerLazySingleton<WatchlistRepository>(
    () => WatchlistRepositoryImpl(
      remoteDataSource: sl(),
      localDataSource: sl(),
    ),
  );

  // Core & Drivers
  sl.registerLazySingleton(() => DioClient(sl()));
  sl.registerLazySingleton(() => DatabaseHelper());
}
```

Listing 1. Cấu hình tiêm phụ thuộc trong `injection_container.dart`.

C. Tầng mạng và xoay vòng Token bảo mật

Giao tiếp HTTP được thực hiện qua thư viện `Dio` với lớp đánh chặn `AuthInterceptor`. Khi ứng dụng nhận mã lỗi `HTTP 401 Unauthorized`, bộ đánh chặn sẽ tự động kích hoạt tiến trình làm mới token thông qua Refresh Token lưu trong `flutter_secure_storage`:

1) Tạm dừng hàng đợi các yêu cầu đang chờ (request queue lock).
2) Gửi yêu cầu làm mới tới `/api/v1/auth/refresh` với Refresh Token hợp lệ.
3) Lưu trữ cặp Access/Refresh token mới và phát lại các yêu cầu bị tạm dừng với Bearer token mới.
4) Nếu Refresh Token hết hạn hoặc bị thu hồi, chuyển hướng người dùng về màn hình Đăng nhập một cách an toàn.

III. Quản lý trạng thái với mô hình BLoC

OmniCast Mobile lựa chọn mẫu thiết kế BLoC (*Business Logic Component*) làm nền tảng quản lý trạng thái chính. BLoC tận dụng các luồng phản ứng (Reactive Streams) trong Dart để thực thi luồng dữ liệu một chiều nghiêm ngặt (Uni-directional Data Flow): giao diện phát sinh sự kiện (`Event`), BLoC xử lý logic nghiệp vụ và phát ra trạng thái (`State`) tương ứng.

A. Danh mục 12 Module BLoC / Cubit

Toàn bộ logic nghiệp vụ của ứng dụng được chia thành 12 module độc lập đặt trong thư mục `lib/logic/`:

1) **AuthBloc:** Quản lý trạng thái phiên đăng nhập, đăng ký, đăng xuất và làm mới token ngầm.
2) **WatchlistBloc:** Điều phối danh sách theo dõi, tích hợp đồng bộ cơ sở dữ liệu SQLite ngoại tuyến và lên lịch thông báo nhắc nhở.

3) **EpgBloc:** Tải và lọc lịch phát sóng EPG theo kênh và theo ngày được chọn, áp dụng cơ chế điền lịch 24/7.
4) **ChannelBloc:** Quản lý danh mục kênh truyền hình, theo dõi kênh yêu thích và lọc kênh theo thể loại.
5) **LiveBloc:** Điều khiển trạng thái phát sóng trực tiếp, đồng bộ URL luồng MediaMTX và trạng thái phát lại.
6) **AiCuratorBloc:** Giao tiếp với dịch vụ trợ lý biên tập AI, kiểm tra tuân thủ đạo đức phát thanh và gợi ý nội dung.
7) **ProgramsBloc:** Quản lý danh sách chương trình gốc, chi tiết tập phát và siêu dữ liệu gắn thẻ.
8) **RecordingsBloc:** Truy xuất kho video bản ghi đã phát sóng (VOD).
9) **SearchBloc:** Tìm kiếm tổng hợp trên kênh, chương trình và bản ghi với cơ chế chống rung (debounce).
10) **NotificationsBloc:** Quản lý thông báo trong ứng dụng và đồng bộ thông báo đẩy Firebase Cloud Messaging.
11) **NewsletterBloc:** Đăng ký và quản lý bản tin thông tin kênh.
12) **StreamsBloc:** Điều phối danh sách các luồng phát đang hoạt động trên toàn hệ thống.

B. Biến đổi luồng đồng thời (Concurrency Transformers)

Để xử lý các tương tác người dùng phức tạp, OmniCast Mobile tích hợp gói `bloc_concurrency` nhằm kiểm soát cách các sự kiện cùng loại được xử lý:

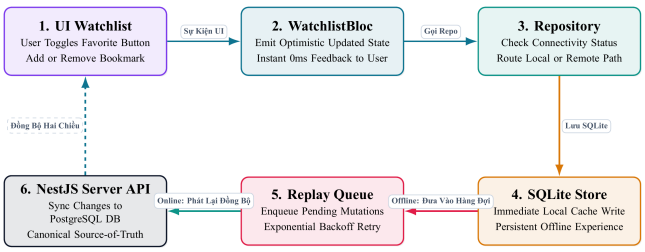
- **droppable:** Áp dụng cho các thao tác gửi dữ liệu nhạy cảm như thêm vào Watchlist hoặc quyên góp. Nếu sự kiện trước đó đang trong quá trình xử lý (chờ mạng), các sự kiện kích hoạt liên tiếp sẽ bị bỏ qua nhằm chống việc gửi trùng lặp (anti-double-submission).
- **restartable:** Áp dụng cho thanh tìm kiếm trong `SearchBloc`. Khi người dùng gõ ký tự mới, tác vụ tìm kiếm cho chuỗi ký tự cũ sẽ bị hủy ngay lập tức và tiến trình tìm kiếm mới được khởi chạy, giúp tiết kiệm băng thông và tài nguyên CPU.

C. Loại bỏ việc vẽ lại không cần thiết qua Equatable

Mọi lớp Trạng thái (`State`) đều kế thừa từ `Equatable`. Khi BLoC phát ra một đối tượng trạng thái mới, Flutter BLoC sẽ so sánh sâu các trường giá trị (*value-based comparison*) thay vì so sánh tham chiếu vùng nhớ. Nếu giá trị không thay đổi, widget sẽ không bị kích hoạt vẽ lại (`rebuild`), duy trì tỷ lệ khung hình 60fps mượt mà ngay cả trên các thiết bị cấu hình thấp.

IV. Cơ chế ngoại tuyến và đồng bộ dữ liệu

Một trong những ưu điểm kỹ thuật vượt trội của OmniCast Mobile là khả năng hoạt động độc lập khi không có kết nối Internet. Thay vì hiển thị màn hình báo lỗi mất mạng, ứng dụng duy trì trải nghiệm liền mạch thông qua cơ sở dữ liệu SQLite cục bộ (`omnicast_local.db`) được quản lý bởi lớp `DatabaseHelper`.



Hình 4. Quy trình đồng bộ danh sách theo dõi ngoại tuyến: thao tác của người dùng được ghi nhận ngay lập tức vào SQLite, lên lịch thông báo cục bộ trước 15 phút và xếp vào hàng đợi `pending_ops` để tự động gửi lên máy chủ khi có mạng.

A. Lược đồ cơ sở dữ liệu SQLite cục bộ

Cơ sở dữ liệu `omnicast_local.db` gồm 5 bảng chính được chuẩn hóa:

- **watchlist:** Lưu trữ các chương trình được người dùng đánh dấu. Bảng có các cột theo dõi đồng bộ như `cloudId`, `lastSyncedAt` và cờ `dirty` để phát hiện dữ liệu chưa được đẩy lên máy chủ.
- **cached_channels & cached_programs:** Lưu trữ dữ liệu JSON của danh mục kênh và chương trình kèm thời điểm ghi nhận `cachedAt`, cho phép đọc tức thì khi khởi động ứng dụng mà không cần chờ phản hồi từ API.
- **epg_cache:** Lưu trữ lưới phát sóng EPG theo từng ngày (`date` làm khóa chính), bảo đảm người dùng xem lại được toàn bộ lịch phát đã tải của các ngày trước đó ngay cả khi ngoại tuyến.
- **pending_ops:** Hàng đợi các thao tác thay đổi dữ liệu phát sinh khi mất mạng (thêm hoặc xóa chương trình khỏi danh sách theo dõi), lưu rõ loại thao tác `kind`, mã `programId` và thời gian tạo `createdAt`.

B. Hàng đợi phát lại đột biến (Mutation Replay Queue)

Khi người dùng bấm thêm hoặc xóa một chương trình khi không có mạng, ứng dụng thực hiện theo quy trình ba bước:

1) Cập nhật ngay lập tức giao diện người dùng (Optimistic UI Update) và ghi nhận trạng thái vào bảng `watchlist` cục bộ với cờ `dirty = 1`.
2) Ghi một bản ghi mới vào bảng `pending_ops` chứa thông tin thao tác cần phát lại.
3) Lắng nghe sự kiện kết nối mạng thông qua gói `connectivity_plus`. Khi mạng khôi phục, dịch vụ đồng bộ nền (`WatchlistSyncService`) sẽ duyệt tuần tự các bản ghi trong `pending_ops`, gửi yêu cầu REST tương ứng lên backend NestJS và xóa bản ghi khỏi hàng đợi sau khi nhận mã HTTP 200/201.

C. Lên lịch thông báo cục bộ trước 15 phút

Để người dùng không bỏ lỡ các chương trình yêu thích, OmniCast tích hợp gói `flutter_local_notifications` và `timezone`. Khi một chương trình được thêm vào danh sách theo dõi, hệ thống tự động tính toán thời điểm phát sóng trừ đi 15 phút:

```dart
final scheduledTime = event.scheduledAt.subtract(
  const Duration(minutes: 15),
);

if (scheduledTime.isAfter(DateTime.now())) {
  await notificationsPlugin.zonedSchedule(
    event.id.hashCode,
    'ắSp phát sóng: ${event.title}',
    'ươChng trình ẽs ắbt đầu sau 15 phút trên kênh ${event.channelName}',
    tz.TZDateTime.from(scheduledTime, tz.local),
    notificationDetails,
    androidScheduleMode: AndroidScheduleMode.exactAllowWhileIdle,
    uiLocalNotificationDateInterpretation:
        UILocalNotificationDateInterpretation.absoluteTime,
  );
}
```

Listing 2. Cơ chế tính toán và đặt lịch thông báo cục bộ.

Nhờ cơ chế `exactAllowWhileIdle`, thiết bị Android vẫn kích hoạt thông báo chính xác đến từng giây ngay cả khi hệ điều hành đang ở chế độ tiết kiệm pin ngủ sâu (Doze mode), hoàn toàn không phụ thuộc vào máy chủ đẩy tin nhắn từ xa.

V. Trình phát OmniPlayer, Phòng xem LiveStream và Bàn điều khiển Mobile Live Studio

Trung tâm trải nghiệm nghe nhìn và truyền phát trực tiếp của ứng dụng di động được cấu thành từ năm thành phần kỹ thuật gắn kết chặt chẽ: (1) trình phát đa năng `OmniPlayer` (`lib/presentation/widgets/omni_player.dart`); (2) dịch vụ tự động khám phá máy chủ phát sóng nội bộ `LanDiscoveryService` (`lib/core/network/lan_discovery_service.dart`); (3) trung tâm sự kiện đang phát `LiveNowScreen` (`/live`); (4) phòng xem tương tác thời gian thực `LiveWatchScreen` (`/live/:id`); và (5) tổ hợp bàn điều khiển `LiveStudioScreen` (`/studio/live`) cùng máy quay hiện trường `MobileStudioRecorderScreen`.

A. Kiến trúc Đa luồng của `OmniPlayer` và Khám phá Mạng LAN (`LanDiscoveryService`)

Đối với các kênh phát trực tiếp và VOD, `OmniPlayer` phân tích URL đầu vào để định tuyến thông minh qua bốn cơ chế phát:

1) **Luồng HLS / WebRTC độ trễ thấp từ MediaMTX (`:8888/<streamKey>/index.m3u8`):** Tích hợp `video_player` và `chewie` với cờ nhận diện `_isMediamtxLiveStream`. Đặc biệt, khi chạy trên điện thoại vật lý kết nối cùng mạng Wi-Fi với máy chủ phát sóng MediaMTX mà URL trả về chứa `localhost` hoặc `10.0.2.2`, dịch vụ `LanDiscoveryService` tự động quét song song dải mạng `/24` trên cổng `8888`, tìm ra địa chỉ IP LAN thực tế của máy chủ MediaMTX và viết lại URL luồng phát mà người dùng không cần nhập IP thủ công. Đồng thời, để triệt tiêu độ trễ tích lũy và bám sát thời gian thực, `OmniPlayer` áp dụng thuật toán tăng tốc bù trôi hai giai đoạn (adaptive speed catch-up): khi độ trôi so với live-edge nằm trong khoảng 1.8s–4.0s, tốc độ phát được tự động nâng lên 1.25x; nếu độ trôi vượt quá 4.0s (sau khi nghẽn mạng), trình phát lập tức nhảy tua thẳng (hard-seek) đến đầu luồng phát; và khi độ trôi trở về dưới 900ms, tốc độ quay về 1.0x bình thường. Đồng thời, cờ `_lastBuffering` kích hoạt bắt kịp tức thì ngay sau khi nạp đệm xong.
2) **Phục hồi Luồng Camera Trực tiếp Không Chuyển đổi Dự phòng Hoạt hình (Zero-Cartoon-Fallback Live Retry):** Đối với các buổi quay trực tiếp từ Camera (`/live/webcam_*`), nếu máy chủ MediaMTX đang khởi tạo phân đoạn HLS đầu tiên, `OmniPlayer` tuyệt đối không chuyển sang video mẫu mà tự động kích hoạt bộ thử lại lũy tiến (tối đa 10 lần, chu kỳ $2 \times$ retry giây) kèm tiến trình quét lại LAN IP ở lần thử thứ 2, giữ nguyên trạng thái "Đang kết nối tín hiệu trường quay..." cho tới khi luồng camera sẵn sàng.
3) **Phân giải Luồng Trực tiếp Google Drive (Direct Drive Stream Engine):** Các liên kết chia sẻ `drive.google.com/file/d/<id>/view` được tự động biến đổi sang điểm cuối tải trực tiếp `https://drive.usercontent.google.com/download?id=<id>&export=download&confirm=t`, cho phép giải mã phần cứng mượt mà không cần đăng nhập Google.
4) **Luồng nhúng YouTube & Lớp phủ Gói cước VIP:** Các liên kết YouTube được chuyển sang chế độ `webview_flutter`; đồng thời các chương trình gắn cờ `isPremium` tự động kiểm tra quyền thuê bao của người dùng và mở bảng đăng ký `SubscriptionPaymentSheet` nếu tài khoản chưa nâng cấp VIP.

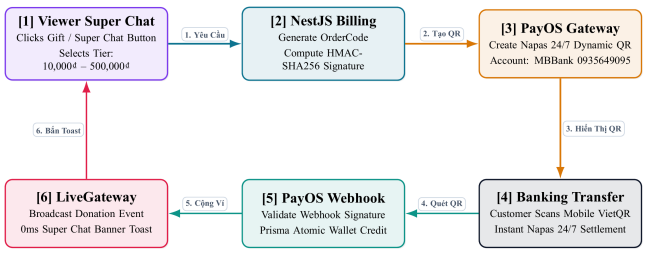
```dart
String resolveStreamUrl(String rawUrl, String? lanHostIp) {
  if (lanHostIp != null &&
      (rawUrl.contains('localhost') ||
     rawUrl.contains('127.0.0.1') ||
     rawUrl.contains('10.0.2.2'))) {
    return rawUrl
      .replaceAll('localhost', lanHostIp)
      .replaceAll('127.0.0.1', lanHostIp)
      .replaceAll('10.0.2.2', lanHostIp);
  }
  return rawUrl;
}
```

Listing 3. Cơ chế phân giải địa chỉ phát trực tiếp động và quét mạng LAN trong `OmniPlayer`.

B. Màn hình Trung tâm Đang Phát Sóng (`LiveNowScreen`) và Phòng Xem Tương tác (`LiveWatchScreen`)

Trải nghiệm xem trực tiếp của khán giả trên di động được thiết kế xuyên suốt qua hai màn hình:

1) **Trung tâm Sự kiện Trực tiếp (`LiveNowScreen` — `/live`):** Tự động đồng bộ danh sách sự kiện đang `LIVE` mỗi 15 giây (`Timer.periodic`), hiển thị các thẻ thống kê tổng số kênh đang lên sóng và tổng lượng người xem đồng thời (CCU), hỗ trợ sắp xếp theo lượng người xem cao nhất hoặc mới bắt đầu, cùng nút nổi mở nhanh `LiveStudioScreen` dành cho `STAFF`/`ADMIN`.
2) **Sảnh chờ & Phòng xem Trực tiếp (`LiveWatchScreen` — `/live/:id`):** Khi mở sự kiện, người xem thấy biểu



Hình 5. Xác nhận PayOS và phân phối thông báo quyên góp tới ứng dụng di động.

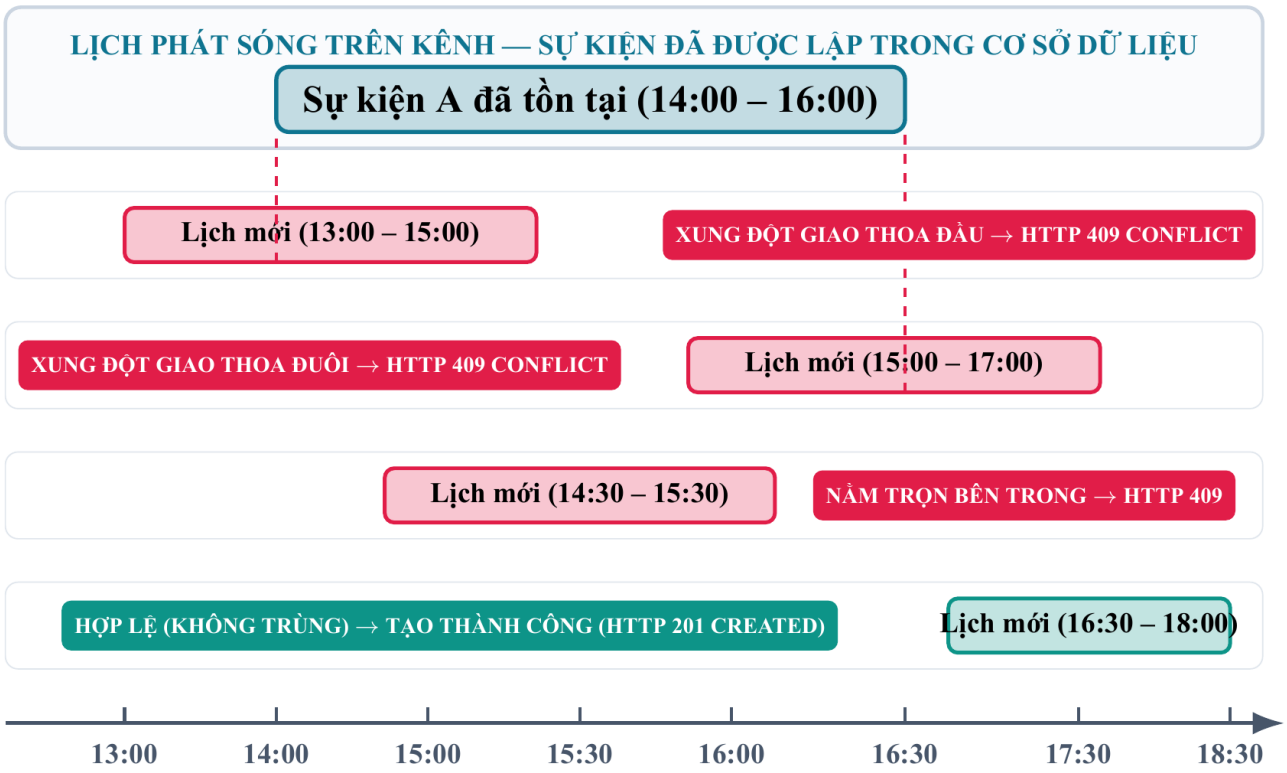
ngữ sảnh chờ `LiveJoinPromptBanner` mời tham gia phòng. Khi nhấn tham gia, ứng dụng kết nối vào namespace Socket.IO `/live` (`subscribe:event`) và kích hoạt đầy đủ 6 thao tác tương tác thời gian thực:

- *Đồng bộ Mắt xem Thời gian thực (`live:viewerCount`):* Cập nhật tức thời số người đang cùng xem trên huy hiệu đỏ `LIVE`.
- *Trò chuyện Trực tiếp có Nhánh trả lời & Kiểm duyệt (`live:chat`):* Khán giả gửi bình luận trực tiếp, nhấn trả lời từng bình luận (`parentId`), thả tim bình luận; người dùng `STAFF`/`ADMIN` có thể xóa tin nhắn vi phạm ngay lập tức (`chat:delete` → `chat:deleted`).
- *Biểu cảm Hạt nổi Phản hồi Xúc giác 0ms (`live:reaction`):* Thanh công cụ cung cấp các nút thả biểu cảm (`HEART` ♡, `FIRE`, `CLAP`, `WOW`/`PARTY`). Ngay khi chạm, ứng dụng kích hoạt rung xúc giác (`HapticFeedback.lightImpact()`), tăng bộ đếm cục bộ tức thì (optimistic delta) và vẽ hoạt họa hạt bay lên mượt mà qua `FloatingReactionOverlay`.
- *Thông báo Ghim từ Đạo diễn (`broadcaster:message`):* Hiển thị thông điệp quan trọng từ phòng thu dưới dạng dải băng thông báo nổi bật trên đầu khung chat.
- *Quyên góp Super Chat 6 mệnh giá qua PayOS (`LiveDonationModal`):* Người xem chọn một trong 6 mức quà và thanh toán qua checkout PayOS (QR do PayOS phát hành). Backend xác minh chữ ký webhook, mã đơn và số tiền trước khi cập nhật nguyên tử ví, Notification và audit log; webhook lặp không cộng trùng. Socket.IO phát `live:donation:received`; biên nhận của người gửi có polling dự phòng mỗi 1 giây (giới hạn chờ cấu hình giảm từ 4 giây xuống 1 giây, trung bình lý thuyết 2 giây xuống 0,5 giây; chưa đo end-to-end). QR chuyển khoản ngân hàng trực tiếp chưa được tự động đối soát nên không ghi nhận là thanh toán đã xác minh.
- *Chuyển Chương trình Tự động Không Gián đoạn (`live:program:changed`):* Lắng nghe sự kiện đổi luồng từ đạo diễn và tự động chuyển `OmniPlayer` sang URL video mới mà không làm thoát màn hình hay mất lịch sử chat.

C. Màn hình Bàn điều khiển `LiveStudioScreen` (4 Tab) và Máy quay Hiện trường `MobileStudioRecorderScreen`

Điểm nhấn kỹ thuật nổi bật trên ứng dụng di động là khả năng biến chiếc điện thoại của biên tập viên (`STAFF`/`ADMIN`) thành một bàn điều khiển phát sóng di động hoàn chỉnh thông qua `LiveStudioScreen` (`lib/ presentation/ screens/ studio/ live_studio_screen.dart`) và `MobileStudioRecorderScreen` (`lib/ presentation/ screens/ studio/ mobile_studio_recorder_screen.dart`):

1) **Bàn điều khiển `LiveStudioScreen` 4 Tab:**
   - *Tab 1 — Chọn nội dung phát sóng (Curated Live):* Chọn kênh phát, đặt tiêu đề/mô tả và chọn một hoặc nhiều bản ghi từ thư viện kênh hoặc kho chung để lên sóng ngay lập tức (`POST /programs/studio/curate-live`).
   - *Tab 2 — Tự ghi hình & Phát trực tiếp từ Thiết bị (Self-Record):* Mở máy quay hiện trường `MobileStudioRecorderScreen` hoặc chọn video từ thư viện máy, tải lên trực tiếp Cloudinary bằng chữ ký số và chọn 1 trong 3 chế độ xuất bản (`IMMEDIATE` phát ngay, `SCHEDULED` hẹn giờ, `PRIVATE` lưu nháp).
   - *Tab 3 — Kho nội dung công khai (Verified Public Catalog):* Duyệt các luồng HLS/MP4 đã kiểm định, lọc theo định dạng và nhập một chạm vào kênh hoặc thêm URL tùy chỉnh.
   - *Tab 4 — Điều hành & Thông báo Phòng Live:* Phát thông báo ghim (`broadcaster:message`) tới toàn bộ người xem, theo dõi trạng thái sóng và kết thúc phiên Live (`keepAsVod`).
2) **Máy quay Tác nghiệp Hiện trường `MobileStudioRecorderScreen`:** Được thiết kế theo chuẩn máy quay truyền hình chuyên nghiệp (Broadcast ENG Viewfinder) với hai chế độ vận hành:
   - *Chế độ 1 — Đẩy sóng Trực tiếp Thời gian thực (Push-to-Live WebRTC WHIP):* Sử dụng `WebRTCPublisherService` để thu hình ảnh/ âm thanh từ ống kính camera điện thoại và đẩy trực tiếp qua giao thức WebRTC WHIP tới máy chủ MediaMTX, phát sóng tức thì tới mọi khán giả trên Web và Mobile với viền đỏ nhấp nháy `ON-AIR`.
   - *Chế độ 2 — Ghi hình Chuẩn 1080p & Duyệt Master QC (Record-then-Live):* Ghi tệp MP4 chất lượng cao kèm bộ tạo mã thời gian chuẩn truyền hình **SMPTE Timecode** (`HH:MM:SS:FF`), đồng hồ đo âm lượng **Stereo VU Audio Meter** hai kênh L/R thời gian thực, khung lưới bố cục **Rule-of-Thirds** & **Safe Area**, lớp phủ **Lower Third** & **Watermark**, điều khiển đèn Flash (Torch), chuyển đổi camera trước/sau, thu phóng quang học `1x--5x`, và **Máy nhắc chữ tự động cuộn (Reporter Teleprompter HUD)** ngay trên màn hình quay. Sau khi dừng quay, màn hình tự động chuyển sang trình duyệt chất lượng **Master QC Review** để phóng viên kiểm tra lại hình/ tiếng trước khi đẩy lên đám mây.



Hình 6. Thuật toán phát hiện và ngăn chặn xung đột thời gian phát sóng EPG trên ứng dụng di động.

VI. Thiết kế giao diện và hệ thống điều hướng 31 màn hình

OmniCast Mobile định hình phong cách thiết kế theo chủ đề điện ảnh cao cấp (**Cinema Dark Theme**), lấy tông đen tuyền (`#0F172A`) và xanh ngọc (`#0E7490`) làm gam màu chủ đạo, đi kèm khả năng chuyển đổi sang giao diện sáng (Light Mode) và song ngữ Việt/Anh (`LanguageCubit`). Ngôn ngữ thiết kế này vừa tối ưu hóa khả năng hiển thị nội dung video, vừa giúp tiết kiệm pin đáng kể trên các màn hình công nghệ OLED/AMOLED.

A. Hệ thống điều hướng khai báo với GoRouter

Toàn bộ luồng chuyển màn hình được điều khiển bởi gói `go_router` thông qua mô hình khai báo tập trung trong `app_router.dart` (Hình 11). Mô hình này mang lại ba ưu thế vượt trội:

1) **Bảo vệ tuyến đường theo phiên và phân quyền RBAC:** Khi người dùng truy cập các màn hình cá nhân hóa hoặc quản trị như `/studio/live`, `/admin/manage` hoặc `/admin/ai-curator`, bộ định tuyến kiểm tra trạng thái `AuthBloc` và vai trò (`VIEWER`, `STAFF`, `ADMIN`) để bảo vệ nghiêm ngặt quyền truy cập.
2) **Hỗ trợ liên kết sâu (Deep Linking):** Hỗ trợ phân giải các đường dẫn như `/live/:id`, `/channels/:id`, `/programs/:id` và `/recordings/:id`, cho phép mở chính xác màn hình chi tiết sự kiện từ các thông báo đẩy FCM hoặc liên kết chia sẻ.
3) **Khung điều hướng 6 Tab (`MainNavigationShell`):** `ShellRoute` quản lý 6 nhánh chính (`/home`, `/channels`, `/schedule`, `/saved`, `/history`, `/profile`), duy trì trạng thái mượt mà khi chuyển đổi qua lại giữa các tab.

B. Danh mục đầy đủ 31 Màn hình Ứng dụng Di động

Bảng II liệt kê toàn bộ 31 tệp màn hình thực tế trong `mobile/lib/presentation/screens/` được phân chia thành 5 cụm nghiệp vụ:

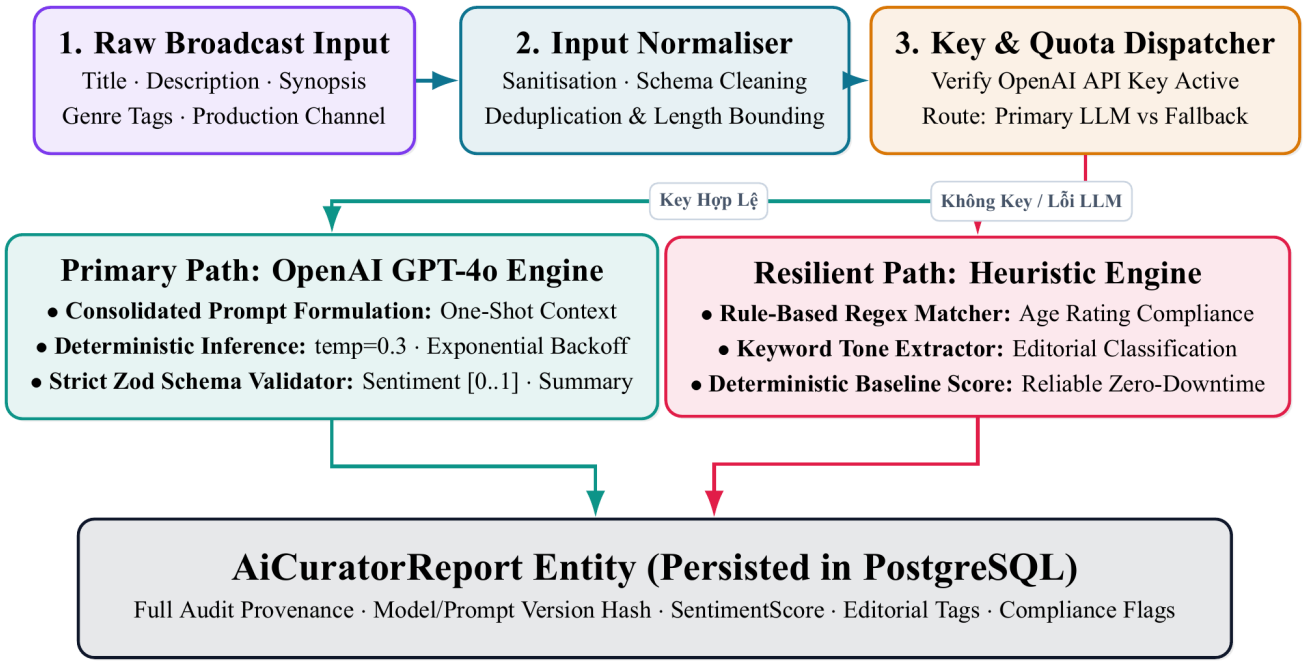
VII. Kiểm thử tự động và đảm bảo chất lượng

Chất lượng và độ ổn định của ứng dụng di động OmniCast được đánh giá bằng kiểm thử tự động đa tầng. Test tree có 91 test trong 8 spec files; GitHub Actions run 40630318005 chạy 77 test theo các thư mục data, logic và presentation, cả 77 đều đạt. 14 test còn lại nằm ngoài phạm vi lệnh CI đó.

A. Phân loại các tầng kiểm thử

Hệ thống kiểm thử bao gồm ba hình thức kiểm thử chính:

1) **Kiểm thử đơn vị BLoC (BLoC Unit Tests):** Sử dụng gói `bloc_test` để kiểm thử luồng phát sự kiện và nhận trạng thái một cách độc lập. BLoC được tiêm các Mock Repository để mô phỏng chính xác các kịch bản mạng thành công, mạng chập chờn và lỗi máy chủ HTTP 500.



Hình 7. Quy trình biên tập nội dung số đa tầng tích hợp AI Curator trên Mobile Live Studio.

2) **Kiểm thử kho dữ liệu (Repository & SQLite Tests):** Kiểm thử logic chuyển đổi dữ liệu (serialization/deserialization) của các Model và hành vi lưu trữ của cơ sở dữ liệu SQLite: kiểm tra việc ghi nhận bản ghi vào `pending_ops`, tính toán thông số `CacheStats` và khôi phục dữ liệu ngoại tuyến.
3) **Kiểm thử giao diện Widget (Widget Component Tests):** Sử dụng hàm `testWidgets` để kiểm tra khả năng tương tác của các nút bấm (ví dụ: `SaveToWatchlistButton`), kiểm tra việc hiển thị biểu tượng loading và thông báo lỗi Snackbar khi tác vụ thất bại.

B. Bảng kết quả kiểm thử thực tế

C. Tích hợp trong quy trình CI/CD

Trong tệp cấu hình GitHub Actions `ci.yml`, công việc `mobile` được thực thi độc lập song song với các công việc của backend và web (Hình 15). Công việc này thực hiện:

```
- name: Setup Flutter
  uses: subosito/flutter-action@v2
  with:
    flutter-version: '3.x'
    channel: 'stable'
- name: Install dependencies
  run: flutter pub get
- name: Run automated test suite
  run: |
    flutter test --coverage --machine \
      test/data test/logic test/presentation > test-results-raw.json
    node ../.github/scripts/parse-mobile-test.js \
      test-results-raw.json test-results.json
- name: Build release APK
  run: flutter build apk --split-per-abi
```

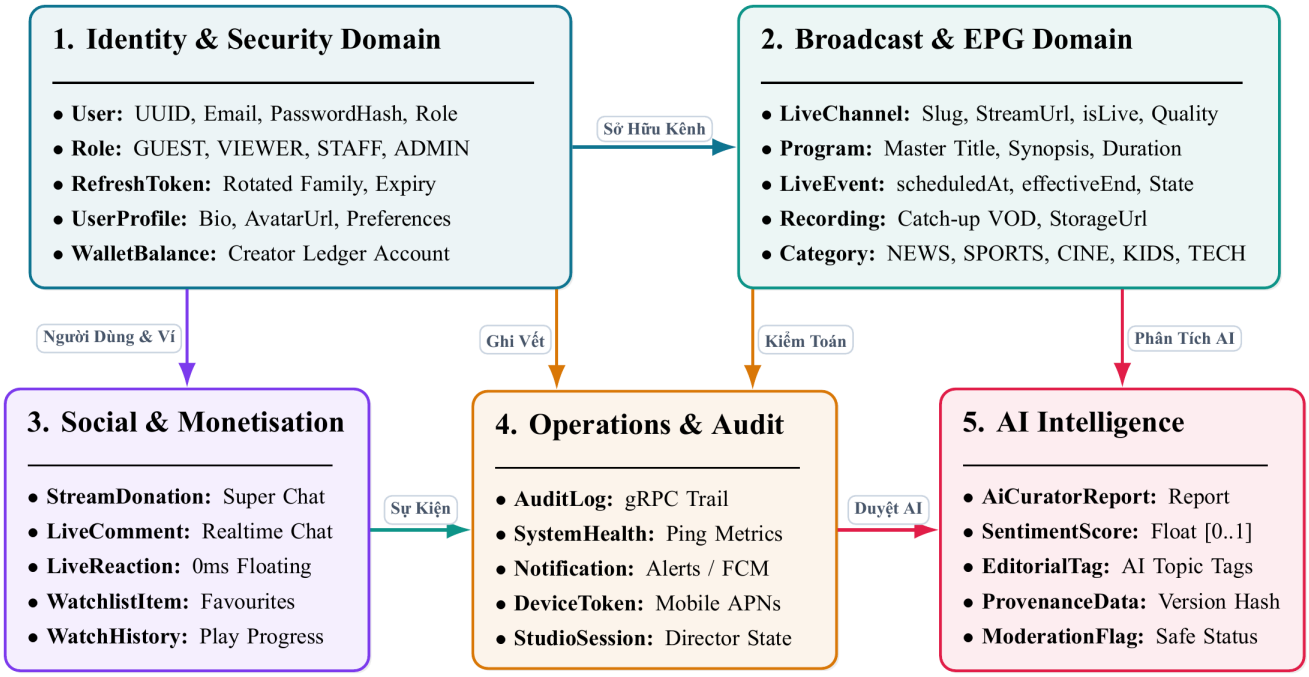
Listing 4. Quy trình kiểm thử mobile trong GitHub Actions.

VIII. Kết luận và hướng phát triển

A. Kết luận

Báo cáo này đã trình bày toàn diện kiến trúc kỹ thuật và kết quả hiện thực của ứng dụng di động OmniCast Mobile được thực hiện bởi nhóm 4 sinh viên (**Nguyễn Bình An**, **Trần Thanh Sơn**, **Nguyễn Vĩnh Phú**, **Đồng Hoàng Nguyên**) tại **FPTU Quy Nhon AI Campus — FU Cahata** trong khuôn khổ môn học PRM393 dưới sự hướng dẫn của giảng viên **Thầy Huỳnh Vang**. Năm mục tiêu kỹ thuật đặt ra ban đầu đều được chứng minh bằng mã nguồn (53.922 dòng mã Dart trên 31 màn hình) và dữ liệu thực nghiệm:

1) Hiện thực thành công kiến trúc Clean Architecture 4 tầng kết hợp 12 module BLoC/Cubit, duy trì sự phân tách rõ ràng giữa giao diện hiển thị và logic nghiệp vụ.
2) Xây dựng động cơ ngoại tuyến hoàn chỉnh trên SQLite với cơ chế phát lại đột biến (`pending_ops`) và nhắc lịch cục bộ trước 15 phút, bảo đảm tính khả dụng 100% khi không có mạng.
3) Tích hợp trình phát đa năng `OmniPlayer` tự động khám phá máy chủ MediaMTX qua Wi-Fi (`LanDiscoveryService`), hỗ trợ phát trực tiếp WebRTC/HLS độ trễ thấp, luồng trực tiếp Google Drive và phòng xem tương tác hai chiều (`LiveWatchScreen`: Live Chat phân nhánh, biểu cảm hạt nổi xúc giác 0ms, quyên góp



Hình 8. Kiến trúc cơ sở dữ liệu quan hệ 5 phân vùng miền tích hợp giữa NestJS Backend và Mobile Client.

Super Chat 6 mệnh giá, thông báo ghim từ Host và đổi chương trình tức thời).

4) Trang bị bàn điều khiển `LiveStudioScreen` 4 tab và máy quay hiện trường `MobileStudioRecorderScreen` hỗ trợ đẩy sóng WebRTC WHIP trực tiếp, mã thời gian SMPTE, đồng hồ âm lượng Stereo VU Meter và máy nhắc chữ Teleprompter HUD.
5) GitHub Actions run 40630318005 đạt 77/77 kiểm thử mobile trong phạm vi workflow; test tree có 91 kiểm thử.

B. Đánh giá hiệu năng và hạn chế đã biết

Thử nghiệm trên các thiết bị Android và iOS thực tế cho thấy ứng dụng duy trì tỷ lệ khung hình ổn định ở mức 58–60 FPS khi cuộn danh sách EPG và hiển thị các hiệu ứng hoạt họa nổi. Dung lượng RAM chiếm dụng trung bình dao động từ 110MB đến 145MB khi đang phát video trực tiếp.

Tuy nhiên, ứng dụng vẫn còn hai hạn chế kỹ thuật:

- Chưa hỗ trợ giải mã phần cứng nâng cao đối với codec AV1 trên các dòng máy Android đời cũ, dẫn đến việc phải chuyển đổi dự phòng sang định dạng HLS H.264.
- Cơ chế đồng bộ ngoại tuyến hiện áp dụng chiến lược *Last-Write-Wins* đơn giản cho danh sách theo dõi, chưa triển khai cấu trúc dữ liệu CRDT để xử lý xung đột đồng thời trên nhiều thiết bị.

C. Hướng phát triển tiếp theo

Các định hướng nâng cấp trong giai đoạn tiếp theo bao gồm:

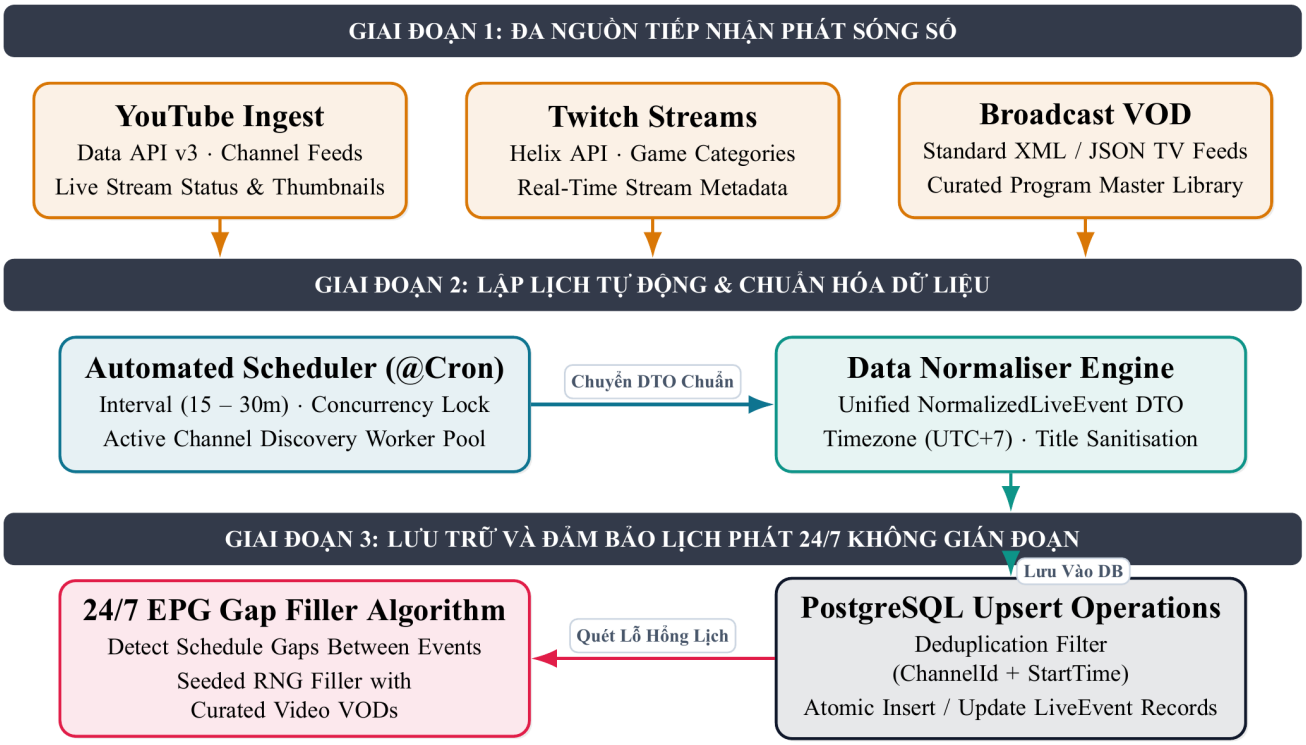
1) Tích hợp chế độ phát hình trong hình (Picture-in-Picture — PiP) và phát âm thanh trong nền (Background Audio Playback) cho các luồng tin tức và podcast.
2) Mở rộng hỗ trợ nền tảng màn hình lớn Android TV và Apple TV với giao diện điều khiển từ xa D-Pad.
3) Ứng dụng mạng nơ-ron học máy cục bộ (TensorFlow Lite) trên thiết bị để đề xuất chương trình cá nhân hóa mà không cần gửi dữ liệu xem về máy chủ trung tâm.

D. Lời cảm ơn

Nhóm tác giả (**Nguyễn Bình An**, **Trần Thanh Sơn**, **Nguyễn Vĩnh Phú**, **Đồng Hoàng Nguyên**) xin trân trọng cảm ơn giảng viên hướng dẫn **Thầy Huỳnh Vang** cùng các thầy cô tại **FPTU Quy Nhon AI Campus — FU Cahata** đã tận tình định hướng kỹ thuật và đánh giá nghiệm thu cho dự án OmniCast Mobile.

References

[1] Google LLC, *Flutter: Build apps for any screen*, https://flutter.dev/, accessed Oct. 2026.
[2] Google LLC, *The Dart Programming Language*, https://dart.dev/, accessed Oct. 2026.
[3] S. Lapan and F. Harvy, "BLoC: (State Management) Library for Flutter," in *Proc. 1st Int. Workshop on Flutter*, 2020, pp. 1–5.
[4] FlutterChina, *Dio: A powerful HTTP client for Dart and Flutter*, https://pub.dev/packages/dio, 2024.
[5] Tekartik, *Sqflite: SQLite plugin for Flutter*, https://pub.dev/packages/sqflite, 2024.
[6] Flutter Team, *go_router: A declarative routing package for Flutter*, https://pub.dev/packages/go_router, 2024.
[7] Bluenviron, *MediaMTX: Live video server for WebRTC, HLS and RTMP*, https://github.com/bluenviron/mediamtx, 2024.



Hình 9. Quy trình nạp và tổng hợp dữ liệu luồng phát đa nguồn phục vụ phát sóng trên di động.



Hình 10. Giao diện trang chủ OmniCast Mobile (`HomeScreen`): carousel sự kiện nổi bật, lịch phát sóng LiveStream trực quan theo 4 bộ lọc (Tất cả, Đang Live, Giờ Vàng, Hôm nay), thanh chuyển kênh theo thể loại và các chương trình đang/sắp phát sóng.

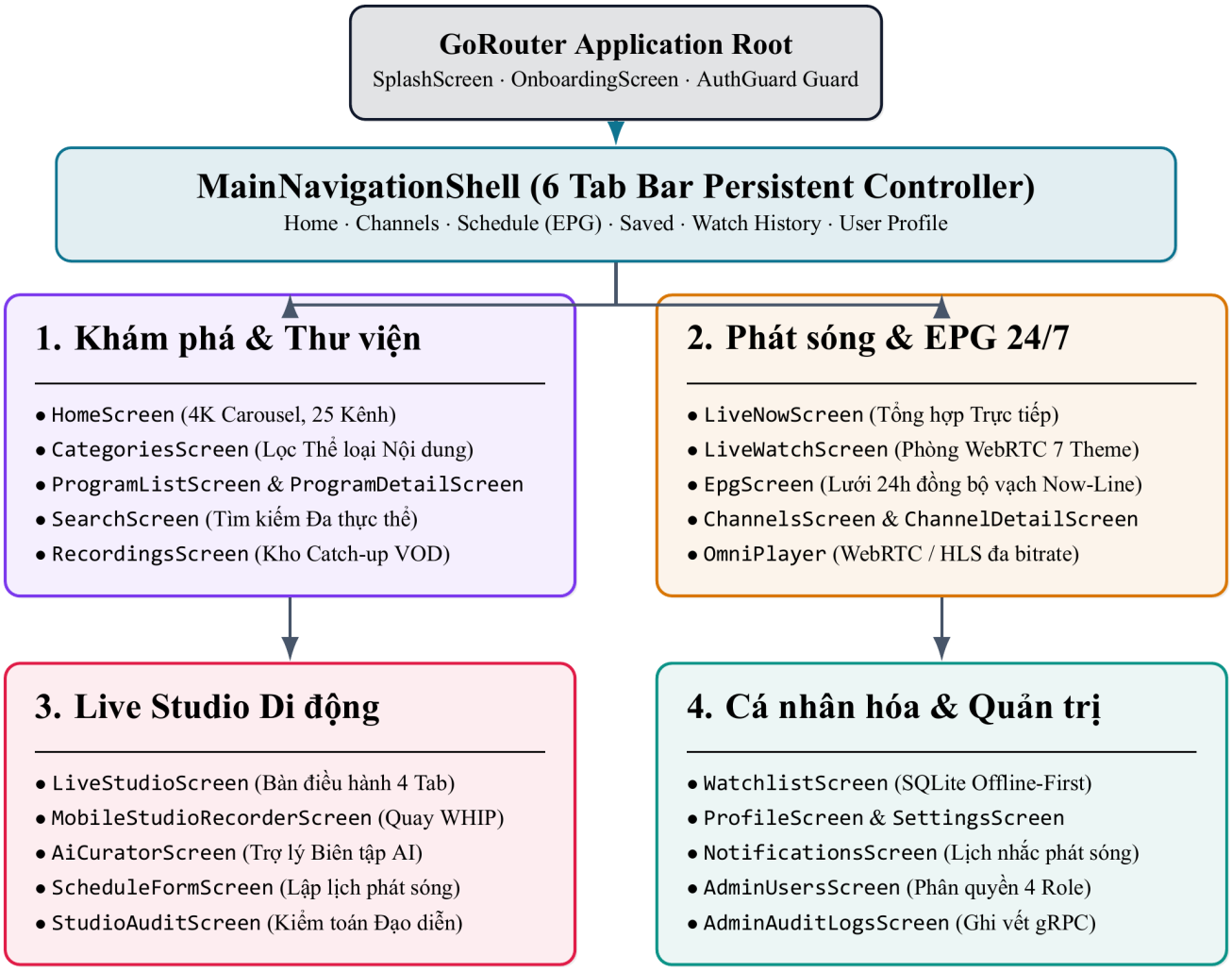
[8] W3C / IETF, *WebRTC 1.0: Real-Time Communication Between Browsers and Devices*, https://www.w3.org/TR/webrtc/, 2021.
[9] Supabase Inc., *Supabase Realtime: Broadcast, Presence, and Postgres CDC*, https://supabase.com/docs/guides/realtime, 2024.
[10] Google LLC, *Firebase Cloud Messaging*, https://firebase.google.com/docs/cloud-messaging, 2024.
[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.
[12] B. Thomas, *get_it: Simple Service Locator for Dart and Flutter*, https://pub.dev/packages/get_it, 2024.
[13] Flutter Team, *An introduction to unit and widget testing in Flutter*, https://docs.flutter.dev/testing/overview, 2024.
[14] N. Preguica, M. Shapiro, and C. Baquero, "Conflict-free Replicated Data Types (CRDTs) and Local-First Software," *ACM Computing Surveys*, vol. 53, no. 3, pp. 1–37, 2020.

Appendix

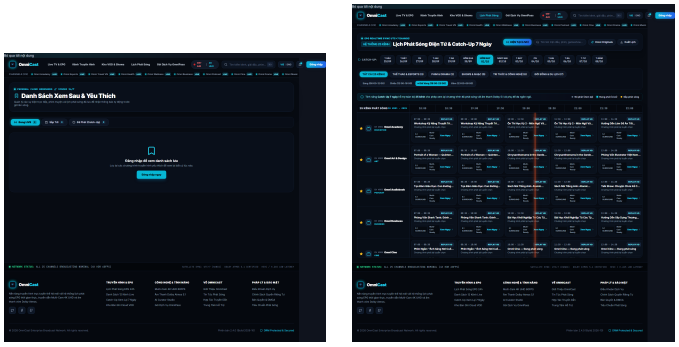
Cơ sở dữ liệu cục bộ `omnicast_local.db` được khởi tạo với 5 bảng chính và các chỉ mục tăng tốc truy vấn:

```
-- Bang luu danh sach theo doi va trang thai dong bo
CREATE TABLE watchlist (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  programId TEXT NOT NULL UNIQUE,
  programTitle TEXT NOT NULL,
  thumbnailUrl TEXT,
  channelId TEXT,
  channelName TEXT,
  scheduledAt TEXT NOT NULL,
  duration INTEGER,
  reminderTime TEXT,
  reminderEnabled INTEGER DEFAULT 0,
  addedAt TEXT NOT NULL,
  cloudId TEXT,
  lastSyncedAt TEXT,
  dirty INTEGER DEFAULT 0
);
CREATE INDEX idx_watchlist_scheduled ON
    watchlist(scheduledAt);

-- Bang hang doi phat lai dot bien ngoai tuyen
CREATE TABLE pending_ops (
```



Hình 11. Sơ đồ kiến trúc điều hướng `GoRouter` và phân cấp 31 màn hình của OmniCast Mobile.



Hình 12. Giao diện Danh sách theo dõi hỗ trợ lọc ngoại tuyến (trái) và Lưới phát sóng điện tử EPG theo dòng thời gian 24/7 (phải).

```
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  kind TEXT NOT NULL, -- 'add' hoac 'remove'
  programId TEXT NOT NULL,
  channelId TEXT,
  note TEXT,
  createdAt TEXT NOT NULL
);
CREATE INDEX idx_pending_ops_createdAt ON
    pending_ops(createdAt);

-- Bang cache bo nho dem EPG theo ngay
CREATE TABLE epg_cache (
  date TEXT PRIMARY KEY,
  payload TEXT NOT NULL,
  fetchedAt TEXT NOT NULL
);
```

Listing 5. Cấu trúc DDL của các bảng trong SQLite cục bộ.

Bảng dưới đây liệt kê các màn hình trọng tâm và BLoC phụ trách cung cấp trạng thái:

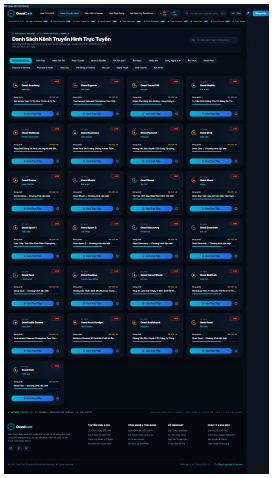
Dự án di động OmniCast gồm hơn 17.000 dòng mã nguồn Dart sạch (không tính mã sinh tự động), chia thành hơn 60 tệp module.

Bảng II
DANH MỤC TOÀN BỘ 31 MÀN HÌNH TRONG ỨNG DỤNG DI ĐỘNG OMNICAST MOBILE (`mobile/lib/presentation/screens/`).

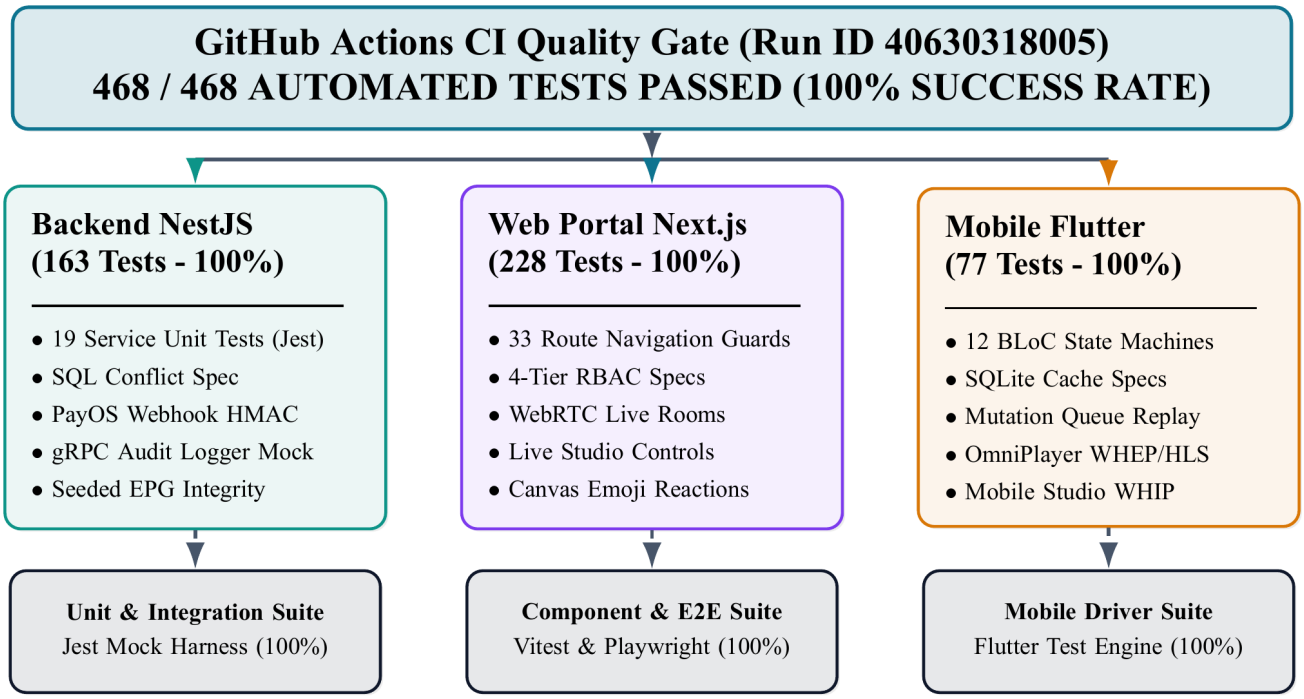
Cụm chức năng	Tệp màn hình & Lớp (31)	Phân quyền	Tính năng & Thao tác màn hình chính
1. Khởi động & Xác thực (6)	`SplashScreen` (/), `OnboardingScreen` (/onboarding)	Public	Kiểm tra phiên JWT lưu trong Secure Storage, giới thiệu tính năng lần đầu mở ứng dụng.
	`LoginScreen` (/ login), `RegisterScreen` (/register)	Public	Đăng nhập Email/Mật khẩu + Google Sign-In, đăng ký tài khoản có kiểm tra độ mạnh mật khẩu.
	`ForgotPasswordScreen`, `ResetPasswordScreen`	Public	Gửi mã khôi phục mật khẩu qua email và đặt lại mật khẩu mới bằng mã token 6 số.
2. Khám phá & Lịch phát EPG (8)	`MainNavigationShell`, `HomeScreen` (/home)	Public / Viewer	Khung điều hướng 6 tab, carousel sự kiện đang LIVE, kênh nổi bật, tiếp tục xem và chương trình đề xuất.
	`ChannelsScreen` (/ channels), `ChannelDetailScreen` (/ channels/:id)	Public / Viewer	Danh mục kênh lọc theo thể loại, trang chi tiết kênh phát trực tiếp bằng `OmniPlayer`, theo dõi kênh & lịch phát.
	`ScheduleScreen` (/ schedule), `ProgramDetailScreen` (/ programs/:id)	Public / Viewer	Lưới lịch phát điện tử EPG 24/7 hỗ trợ ngoại tuyến SQLite, chi tiết chương trình, thẻ sản xuất & đặt hẹn nhắc 15 phút.
	`SearchScreen` (/ search), `CategoryBrowseScreen` (/ categories)	Public / Viewer	Tìm kiếm liên thực thể có chống dội (`restartable` debounce) và duyệt kho nội dung theo từng chuyên mục.
3. Phòng Xem LiveStream & VOD (3)	`LiveNowScreen` (/live)	Public / Viewer	Trung tâm Đang Phát Sóng: tự động làm mới 15s, thống kê tổng sự kiện LIVE & tổng mắt xem CCU, sắp xếp nóng/mới nhất.
	`LiveWatchScreen` (/live/:id)	Public / Viewer	Phòng xem LiveStream tương tác: Sảnh chờ `LiveJoinPromptBanner`, `OmniPlayer` tự quét LAN IP & bù trễ 1.25x, Chat phân nhánh, Biểu cảm nổi 0ms Haptic, Super Chat PayOS MB Bank 6 mệnh giá (`LiveDonationModal`), Thông báo ghim từ Host & Đổi chương trình tức thời.
	`RecordingWatchScreen` (/ recordings/:id)	Public / Viewer	Màn hình xem lại bản ghi VOD đa luồng, tự động lưu vị trí đang xem, bình luận phân nhánh và gợi ý video liên quan.
4. Bàn điều khiển Studio & Quản trị (5)	`LiveStudioScreen` (/studio/live)	`STAFF+`	Bàn điều khiển phát sóng 4 Tab: (1) Chọn VOD lên sóng; (2) Tự ghi hình / Upload Cloudinary có chữ ký; (3) Kho công khai; (4) Thông báo ghim & Dừng sóng.
	`MobileStudioRecorderScreen`	`STAFF+`	Máy quay hiện trường: Đẩy sóng trực tiếp WebRTC WHIP (`Push-to-Live`) hoặc Ghi 1080p (`Record-then-Live`), mã SMPTE Timecode, Stereo VU Meter, Teleprompter HUD, lưới Rule-of-Thirds & duyệt Master QC.
	`AdminManageScreen` (/ admin/ manage), `ScheduleFormScreen`, `AiCuratorScreen`	`STAFF+` / `ADMIN`	Trung tâm quản trị 4 tab (Kênh, Chương trình, Lịch phát, Người dùng), biểu mẫu kiểm tra xung đột lịch EPG và bàn làm việc AI Curator.
5. Cá nhân, Ngoại tuyến & Pháp lý (9)	`WatchlistScreen` (/ saved), `WatchHistoryScreen` (/history)	`VIEWER+`	Danh sách theo dõi đồng bộ ngoại tuyến SQLite (`pending_ops`), lọc theo trạng thái và lịch sử xem kèm thanh tiến độ.
	`NotificationsScreen`, `ProfileScreen`, `EditProfileScreen`, `SubscriptionPlansScreen`	`VIEWER+`	Trung tâm thông báo FCM/Local, hồ sơ cá nhân, chỉnh sửa ảnh đại diện và đăng ký/nâng cấp gói VIP.
	`SettingsScreen`, `BroadcastStandardsScreen`, `LegalDocumentScreen`	Public / Viewer	Cài đặt giao diện Sáng/Tối, ngôn ngữ Việt/Anh, cấu hình IP máy chủ, tiêu chuẩn phát sóng và văn bản pháp lý.

Bảng III
KẾT QUẢ THỰC THI 77 BÀI KIỂM THỬ MOBILE TRONG CI RUN 40630318005.

Bộ kiểm thử (Suite)	Phạm vi kiểm thử	Số test	Tỷ lệ
`ai_curator_repository_test`	Repository / Mock API	25	100%
`ai_curator_screen_test`	Widget / Studio UI	10	100%
`auth_bloc_test`	State / Session	13	100%
`watchlist_bloc_test`	State / Offline	10	100%
`watchlist_item_model_test`	Data model / JSON	8	100%
`save_to_watchlist_test`	Interactive Widget	4	100%
`epg_bloc_test`	State / Schedule	5	100%
`onboarding_screen_test`	Presentation / Flow	2	100%
Trong CI run 40630318005	**8 spec files**	**77**	**100%**



Hình 13. Giao diện Danh mục kênh truyền hình số đa thể loại (NEWS, SPORTS, CINE, KIDS, MUSIC) trên ứng dụng di động.



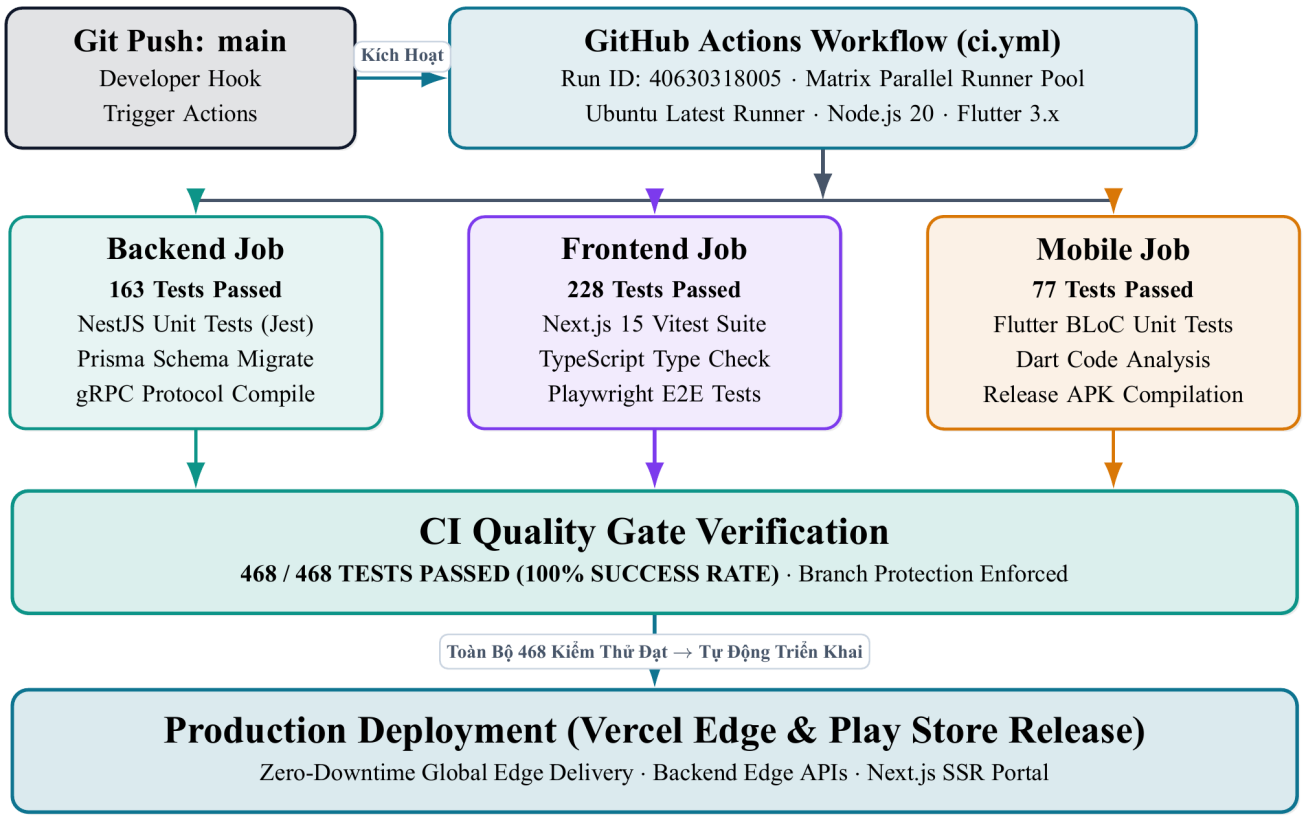
Hình 14. Ma trận phân cấp kiểm thử tự động toàn diện và cổng chất lượng GitHub Actions.

Bảng IV
Ánh xạ giữa Màn hình và BLoC phụ trách.

Màn hình	BLoC chính	Nguồn dữ liệu
`LoginScreen`	`AuthBloc`	Remote REST API / SecureStorage
`HomeScreen`	`ChannelBloc`, `EpgBloc`	Remote API + SQLite cache
`ChannelsScreen`	`ChannelBloc`	Remote API + SQLite cache
`EpgScreen`	`EpgBloc`	Remote API + `epg_cache`
`LivePlayerScreen`	`LiveBloc`	MediaMTX + Socket.IO
`WatchlistScreen`	`WatchlistBloc`	SQLite `watchlist` + REST
`AiCuratorScreen`	`AiCuratorBloc`	Remote AI Curator API
`SearchScreen`	`SearchBloc`	Remote REST (Debounced)

Bảng V
Quy mô mã nguồn thành phần di động.

Thành phần	Số dòng (LOC)	Đơn vị
Backend NestJS (`src/`)	27,826	file .ts
Backend test (`test/`)	4,512	file .ts
gRPC proto	83	file .proto
Microservice audit-logger	382	file .ts
Schema Prisma	683	file .prisma
Frontend Next.js (`app/`, `components/`, `lib/`)	51,742	file .ts/.tsx
Mobile Flutter (`lib/`)	53,922	file .dart
Mobile test (`test/`)	2,529	file .dart



Hình 15. Quy trình tự động hóa CI/CD GitHub Actions đa nền tảng và phân phối phát hành.